E05 Simple Tweens

CRE132 worksheet

What you'll make

Coins and gems that bob gently up and down, each at its own point in the wave, and inventory slots that *pop* in when you pick something up: they grow from nothing, go slightly too big, then settle.

By the end you'll know what a **tween** is (a value moved from A to B over time), how to make one with a coroutine and `Mathf.Lerp`, and how a sine wave makes smooth, endless movement. No extra packages needed.

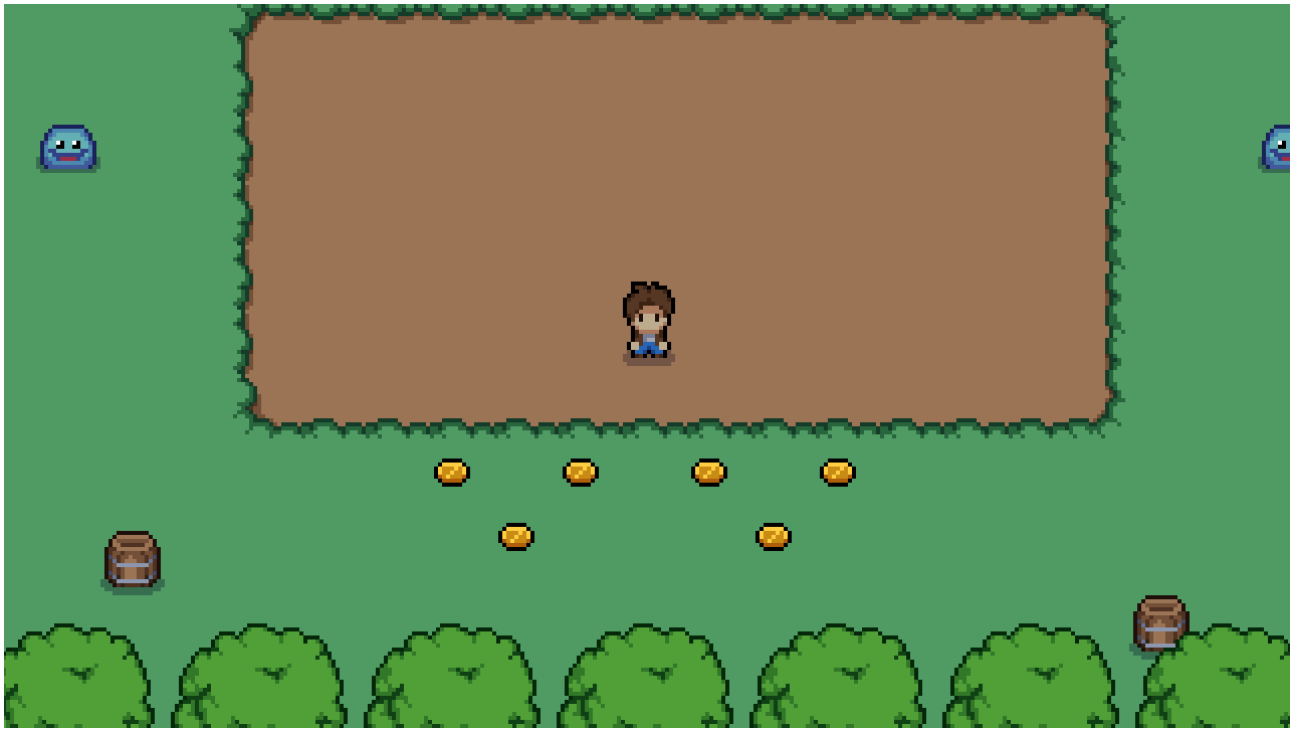


Figure 1: The finished scene: the player among bobbing coins and gems, with the inventory row at the top right.

Before you start

1. Unzip `CRE132_E05_SimpleTweens_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_E05_SimpleTweens_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Effects/E05_SimpleTweens`.
3. Press **Play**. Walk with WASD or the arrows and pick up the coins and gems. They go into the inventory row at the top right (T09's HUD), but nothing moves: the pickups sit still and the slots just appear.

The scene has six **Coin Pickups** and two **Blue Gem Pickups**. The inventory row uses the prefab `Assets/_game/Prefabs/UI/Inventory Slot (Pop In)`, a copy (variant) of T09's slot that's ready for you to add the pop.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Bobbing pickups: Bob

Make a folder `Assets/_game/Scripts/E05_SimpleTweens` and create `Bob` in it:

```
// Makes a pickup bob gently up and down, so it catches the eye. A sine wave added to its
// start height.
using UnityEngine;

public class Bob : MonoBehaviour
{
    [SerializeField] private float height = 0.15f;
    [SerializeField] private float bobsPerSecond = 1f;

    private Vector3 startPosition;
    // A different starting point in the wave for each pickup, so they don't all bob in
    // step.
    private float offset;

    private void Start()
    {
        startPosition = transform.position;
        offset = Random.Range(0f, 10f);
    }

    private void Update()
    {
        float wave = Mathf.Sin((Time.time + offset) * bobsPerSecond * 2f * Mathf.PI);
        transform.position = startPosition + new Vector3(0f, wave * height, 0f);
    }
}
```

- **This isn't a tween but a wave.** `Mathf.Sin` goes smoothly from 0 up to 1, down to −1 and back, for ever. Multiplying by `height` makes it 0.15 units up and down.
- **`* 2f * Mathf.PI`** makes one full wave take one second when `bobsPerSecond` is 1. (A sine wave repeats every 2π.)
- **`startPosition`** is remembered in `Start`, and the wave is added to it every frame. Adding to `transform.position` itself would make the pickup drift away.
- **`offset`** shifts each pickup to a random point in the wave, so they don't all move in step like a marching band.

Add it: in the Hierarchy's search box, type **Pickup** to list all eight coins and gems. Select them all and add **Bob**. Leave **Height** 0.15 and **Bobs Per Second** 1. Clear the search box afterwards.

Test it: press Play. Every pickup bobs, each at its own moment.

2. A pop-in tween: PopIn

Create `PopIn` in the same folder:

```
// Makes a UI element "pop" in when it appears: it grows from nothing, slightly past full
// size, then settles. A tween is just a value moved from A to B over time, here with Lerp
// in a coroutine.
using System.Collections;
using UnityEngine;
```

```
public class PopIn : MonoBehaviour
{
    [SerializeField] private float duration = 0.3f;
    // How far past full size it goes before settling (0.2 = 20 % too big).
    [SerializeField] private float overshoot = 0.2f;

    // OnEnable runs every time the object appears, so it pops every time.
    private void OnEnable()
    {
        StartCoroutine(Pop());
    }

    private IEnumerator Pop()
    {
        float time = 0f;
        while (time < duration)
        {
            // Unscaled, so it still pops while the game is paused (menus).
            time = time + Time.unscaledDeltaTime;
            float progress = Mathf.Clamp01(time / duration);

            // First 70 %: grow from 0 to 1 + overshoot. Last 30 %: shrink back to 1.
            float scale;
            if (progress < 0.7f)
            {
                scale = Mathf.Lerp(0f, 1f + overshoot, progress / 0.7f);
            }
            else
            {
                scale = Mathf.Lerp(1f + overshoot, 1f, (progress - 0.7f) / 0.3f);
            }
            transform.localScale = new Vector3(scale, scale, 1f);
            yield return null;
        }
        transform.localScale = Vector3.one;
    }
}
```

- **`Mathf.Lerp(a, b, t)`** gives a value part way from `a` to `b`: `t` = 0 gives `a`, 1 gives `b`, 0.5 halfway. Move `t` from 0 to 1 over time and you have a tween.
- **`progress`** goes from 0 to 1 over `duration` (0.3 s). For the first 70 % the scale grows from 0 to 1.2; dividing by 0.7 turns that part into its own 0 to 1. The last 30 % shrinks from 1.2 back to 1.
- **Overshoot is easing.** A straight lerp from 0 to 1 looks mechanical. Going past the target and settling back feels springy, like something with weight.
- **`Time.unscaledDeltaTime`** keeps counting when `Time.timeScale` is 0, so the pop still works in a paused menu (S06).
- **The last line** sets the scale to exactly 1, in case the final frame stopped just short.

Add it: double-click the prefab `Prefabs/UI/Inventory Slot (Pop In)` to open it. Select its top object, add **PopIn** (leave **Duration** 0.3 and **Overshoot** 0.2), and close the prefab (the arrow at the top left of the Hierarchy) so it saves.

Test it: press Play and pick up a coin. The slot pops in, slightly too big, then settles. Pick up another: InventoryUI rebuilds the whole row each time the inventory changes, so every slot pops again.

Why a prefab variant? Inventory Slot (Pop In) is a *variant* of T09's **Inventory Slot**: it has everything the original has, plus PopIn. Change the original and the variant follows; other scenes that use the plain slot don't pop.

Check your work

- ☐ Every coin and gem bobs up and down, not all in step.
- ☐ A new inventory slot grows from nothing, goes slightly too big, then settles at normal size.
- ☐ The pickups stay where they were placed and don't drift.
- ☐ Compare with `CRE132_E05_SimpleTweens_Finished` if something behaves differently.

Key points

- **Moving things smoothly between values** is the heart of most UI and pickup animation.
- **`Lerp(a, b, t)` with `t` going from 0 to 1 over time** is all a basic tween needs: a coroutine, a timer and a lerp.
- **Waves** (`Mathf.Sin`) are for movement that never stops; **tweens** are for movement from one place or size to another.
- **Tween libraries** (DOTween, LeanTween, PrimeTween) do the same with less code and many easings, once you know what they're doing.

If something goes wrong

- **The slots don't pop:** PopIn went on the plain **Inventory Slot** prefab, or on an object in the scene, instead of **Inventory Slot (Pop In)**.
- **The pickups float away upwards:** check `startPosition +` is in the line that sets `transform.position`.
- **All the pickups bob together:** `offset` isn't being added to `Time.time`.
- **The slot ends up slightly the wrong size:** the last line, `transform.localScale = Vector3.one;`, is missing.
- **A slot pops on and on, or never appears:** check the `while (time < duration)` loop adds to `time` every frame.

Try it

1. Spin the gems instead of bobbing them: make a copy of Bob that changes the Z angle (`transform.rotation = Quaternion.Euler(0f, 0f, angle);`).
2. Try different easing: **Overshoot** 0.5, or 0 (a plain grow). Change the 0.7 split to 0.5. Which feels best?
3. Pop the Game Over panel in (S06) by adding PopIn to it. Does it still work while the game is paused? Why?
4. Slide a "Gem found!" banner in from the top and out again: lerp its position instead of its scale.